PROJECT DELIVERABLE & SRS IMPLEMENTATION REPORT

รายงานฉบับสมบูรณ์: โครงการ SRS Mini Helpdesk

Mini Helpdesk / Issue Tracking System — ระบบแจ้งปัญหาและติดตามสถานะการดำเนินงานแบบง่าย

ชื่อโปรเจกต์ (Project Name)	ผู้จัดทำ (Developer)	อาจารย์ผู้สอน (Instructor)
SRS Mini Helpdesk	**นาย รัตนภัทร นันทะแสน (681540006005-4)**	**คุณครูวรพจน์ สมมูล**

1. แนวคิด ภาพรวม และวัตถุประสงค์ของระบบ (Project Overview & Objectives)

SRS Mini Helpdesk (Mini Helpdesk / Issue Tracking System) เป็นเว็บแอปพลิเคชันที่พัฒนาขึ้นเพื่อใช้เป็นศูนย์กลางในการรับแจ้งปัญหา ติดตามสถานะการดำเนินงาน และบริหารจัดการงานซ่อมบำรุงอุปกรณ์ด้านเทคโนโลยีสารสนเทศภายในองค์กรหรือหน่วยงานขนาดเล็ก เช่น ห้องเรียน ห้องปฏิบัติการคอมพิวเตอร์ สำนักงาน หรือสาขาวิชา โดยเปลี่ยนจากการแจ้งปัญหาแบบปากเปล่าหรือกระดาษ มาเป็นการบันทึกข้อมูลในรูปแบบ **Ticket** ที่มีรหัสอ้างอิงอัตโนมัติ (เช่น `JJK-0001`) ทำให้ผู้แจ้งสามารถติดตามสถานะได้ตลอดเวลา และช่วยให้ผู้ดูแลระบบสามารถมอบหมายงานรวมถึงดูสถิติภาพรวมผ่านหน้า **Dashboard** ได้อย่างเป็นระบบ

🎯 วัตถุประสงค์ของระบบ (ตามข้อกำหนด SRS หัวข้อที่ 3)

1. พัฒนาเว็บแอปพลิเคชันสำหรับแจ้งและติดตามปัญหาในหน่วยงาน
2. ให้ผู้ใช้ส่งรายการปัญหาพร้อมรายละเอียดและระดับความสำคัญได้อย่างเป็นระบบ
3. ให้ผู้ดูแลและเจ้าหน้าที่จัดการ Ticket มอบหมายงาน และติดตามสถานะได้
4. ฝึกปฏิบัติการพัฒนาเว็บที่มี **CRUD, Workflow, Status Tracking, Search/Filter และ Dashboard**
5. ฝึกการออกแบบ Data Model สำหรับระบบ Issue Tracking ด้วย `LocalStorage`
6. ประยุกต์ใช้ AI Tools ช่วยออกแบบระบบ เขียนโค้ด แก้บั๊ก และจัดทำเอกสารประกอบ

📦 สรุปสิ่งที่ส่งมอบในรายงานฉบับนี้ (ตามข้อกำหนด SRS หัวข้อที่ 23)

- **แนวคิดและฟีเจอร์ของระบบ:** สรุปขอบเขต บทบาทผู้ใช้ และ Workflow การทำงานครบถ้วน
- **โครงสร้างไฟล์และ Data Model:** อธิบายโครงสร้างโค้ดทั้ง 10 ไฟล์ และโครงสร้าง JSON ใน `LocalStorage`
- **คำอธิบายโค้ดหลัก (Core Logic):** เจาะลึก Logic การสร้าง Ticket ID, การเปลี่ยนสถานะ และการกรองข้อมูลจากไฟล์ `app.js`
- **ข้อมูลตัวอย่าง (Sample Data):** ข้อมูล Ticket ตัวอย่าง 10 รายการ (`JJK-0001` – `JJK-0010`)
- **ภาพหน้าจอจริง (Screenshots):** ครบทั้ง 8 หน้าจอหลัก และหน้าจอ Mobile Responsive
- **สรุปการใช้ AI Tools และคู่มือ `README.md`:** อธิบายขั้นตอนการใช้ AI และวิธีติดตั้งใช้งานจริง

2. บทบาทผู้ใช้งาน (User Roles) และขอบเขตของระบบที่พัฒนาจริง

ตามข้อกำหนด SRS หัวข้อที่ 7 (สำหรับเวอร์ชันง่ายที่จำลองบทบาทผ่านเมนูหน้าจอโดยไม่ต้องทำระบบ Login จริง) ระบบได้แยกมุมมองการใช้งานออกเป็น **3 บทบาทหลัก** ผ่านหน้าหลัก (`index.html`) ดังนี้:

1. User View (ผู้ใช้ทั่วไป)

นักศึกษา, อาจารย์, บุคลากร

- กรอกแบบฟอร์มแจ้งปัญหาใหม่ (`user.html`)
- รับหมายเลข Ticket อัตโนมัติ (`JJK-XXXX`)
- ดูรายการปัญหาที่ตนเองแจ้งไว้ (Ticket ของฉัน)
- เปิดดูรายละเอียดและสถานะล่าสุด (`ticket-user.html`)
- **แก้ไข** หัวข้อ/รายละเอียดได้เฉพาะเมื่อสถานะเป็น `Open`
- **ยกเลิก (Cancel)** Ticket ของตนเองได้เมื่อสถานะเป็น `Open`

2. Staff View (เจ้าหน้าที่ IT)

IT Staff 1, IT Staff 2

- เลือกชื่อเจ้าหน้าที่และดูเฉพาะรายการ Ticket ที่ได้รับมอบหมาย (`staff.html`)
- เรียงลำดับงานตามเวลาที่ได้รับมอบหมายล่าสุด (`assignedAt`)
- เปิดดูรายละเอียดปัญหาที่ได้รับมอบหมาย (`staff-ticket.html`)
- อัปเดตสถานะการดำเนินงาน (`Open`, `In Progress`, `Resolved`, `Cancelled`)
- บันทึกหมายเหตุหรือวิธีการแก้ไขปัญหา (`Resolution Note`)

3. Admin View (ผู้ดูแลระบบ)

ผู้ดูแลระบบหลักของหน่วยงาน

- ดู **Dashboard** สรุปยอดรวม แยกตามสถานะ ประเภทปัญหา และความเร่งด่วน (`admin.html`)
- ดูรายการ Ticket ทั้งหมดในระบบ
- ค้นหาด้วย Keyword (Ticket ID, หัวข้อ, รายละเอียด) และกรอง 4 เงื่อนไข
- มอบหมายงานให้ `IT Staff 1`, `IT Staff 2` หรือ `Admin` (`ticket.html`)
- เปลี่ยนสถานะ บันทึกหมายเหตุ และ **ลบ Ticket** ที่ไม่ต้องการ
- เพิ่ม/ลบหมวดหมู่ประเภทปัญหา (`issue-type.html`)

2. ขอบเขตความต้องการของระบบ (Functional & Non-Functional Requirements)

2.1 ความต้องการเชิงฟังก์ชัน (Functional Requirements: FR-01 ถึง FR-16 ตาม SRS หัวข้อที่ 8)

รหัส	รายการความต้องการ	การพัฒนาและการทำงานจริงในโปรเจกต์	สถานะ
FR-01, FR-02	แจ้งปัญหาใหม่และระบุรายละเอียดครบถ้วน	หน้า `user.html` มีฟอร์มกรอกชื่อผู้แจ้ง, อีเมล, หัวข้อ, รายละเอียด, ประเภทปัญหา, สถานที่ และระดับความสำคัญ พร้อมระบบตรวจสอบข้อมูลว่าง (Validation)	ครบถ้วน
FR-03	สร้าง Ticket ID อัตโนมัติ	สร้างรหัสอัตโนมัติในรูปแบบ `JJK-0001`, `JJK-0002` โดยใช้ `padStart(4, "0")` และบันทึกตัวนับ `ticketNumber` ลงใน `localStorage`	ครบถ้วน
FR-04, FR-05	แสดงรายการและดูรายละเอียด Ticket	แสดงรายการใน `user.html`, `admin.html`, `staff.html` และส่งพารามิเตอร์ `?id=JJK-XXXX` ไปแสดงรายละเอียดใน `ticket-user.html`, `ticket.html`, `staff-ticket.html`	ครบถ้วน
FR-06	แก้ไขข้อมูล Ticket	User แก้ไขหัวข้อและรายละเอียดผ่านหน้า `ticket-user.html` ได้เฉพาะกรณีที่สถานะยังเป็น `Open`	ครบถ้วน
FR-07, FR-08	ลบ Ticket และยกเลิก Ticket	Admin ลบ Ticket ออกจากระบบได้ในหน้า `ticket.html` (มี `confirm()` ยืนยัน) และ User กดยกเลิก Ticket ของตนเองเป็น `Cancelled` ได้เมื่อสถานะเป็น `Open`	ครบถ้วน
FR-09, FR-10	เปลี่ยนสถานะและมอบหมายผู้รับผิดชอบ	Admin และ Staff เปลี่ยนสถานะได้ 4 สถานะ; Admin มอบหมายงานให้ `IT Staff 1`, `IT Staff 2`, `Admin` พร้อมประทับเวลา `assignedAt`	ครบถ้วน
FR-11	เพิ่มหมายเหตุการดำเนินงาน	Staff และ Admin บันทึกวิธีการแก้ไขหรือหมายเหตุลงในฟิลด์ `resolutionNote` พร้อมอัปเดตเวลาแก้ไขล่าสุด (`updatedAt`)	ครบถ้วน
FR-12, FR-13	ค้นหาและกรอง Ticket (Search & Filter)	ฟังก์ชัน `filterTickets()` ค้นหาแบบ Real-time จากรหัส, หัวข้อ, รายละเอียด ร่วมกับตัวกรอง 4 ด้าน (ประเภทปัญหา, สถานะ, ความเร่งด่วน, ผู้รับผิดชอบ)	ครบถ้วน
FR-14	Dashboard Summary	แสดงการ์ดสรุปจำนวน Ticket ทั้งหมด แยกตามสถานะ 4 สถานะ พร้อมกล่องสรุปแยกตามประเภทปัญหา 7 หมวด และระดับความเร่งด่วน 4 ระดับ	ครบถ้วน
FR-15, FR-16	Responsive Design & Deploy	รองรับการแสดงผลทั้งคอมพิวเตอร์ แท็บเล็ต และมือถือด้วย `@media (max-width: 600px)` และเป็น Static Web ที่เปิดใช้งานและ Deploy ได้ทันที	ครบถ้วน

2.2 ความต้องการที่ไม่ใช่ฟังก์ชัน (Non-Functional Requirements: NFR-01 ถึง NFR-09 ตาม SRS หัวข้อที่ 9)

รหัส	หัวข้อความต้องการ	แนวทางการออกแบบและพัฒนาในระบบ	สถานะ
NFR-01	ใช้งานง่าย (Usability)	ออกแบบหน้าจอแบบ Card Layout มีปุ่มนำทางกลับหน้าหลัก (←) ชัดเจน แยกสีสถานะด้วย `.status-badge` ทำให้อ่านเข้าใจได้ทันที	ผ่านเกณฑ์
NFR-02, NFR-03	ความถูกต้องและความเร็ว (Accuracy & Speed)	บันทึกและดึงข้อมูลผ่าน `localStorage` แบบ Synchronous ทำให้การค้นหา กรอง เปลี่ยนสถานะ และคำนวณ Dashboard ทำงานเสร็จทันทีโดยไม่หน่วง	ผ่านเกณฑ์
NFR-04	รองรับมือถือ (Mobile Friendly)	กำหนด `viewport` ในทุกหน้า HTML และใช้ CSS Flexbox/Grid ร่วมกับ Media Query ปรับการ์ดและปุ่มให้เต็มความกว้างหน้าจอมือถือ	ผ่านเกณฑ์
NFR-05, NFR-06	ความปลอดภัยและความเป็นส่วนตัวเบื้องต้น	จำกัดสิทธิ์การแก้ไข/ยกเลิกของ User เมื่อ Ticket ไม่ได้อยู่ในสถานะ `Open` และเก็บเฉพาะชื่อกับอีเมลเพื่อใช้ติดตามงานเท่านั้น	ผ่านเกณฑ์
NFR-07, NFR-08, NFR-09	ดูแลโค้ดง่าย, ความน่าเชื่อถือ และความพร้อมใช้งาน	แยกไฟล์ HTML ตามหน้าจอ แยก CSS ไว้ใน `css/style.css` และรวม Logic ไว้ใน `app.js` พร้อม Form Validation ป้องกันข้อมูลไม่ครบถ้วน	ผ่านเกณฑ์

3. สถาปัตยกรรมระบบ โครงสร้างไฟล์ และ Data Model (Architecture & Data Design)

💻 3.1 Technology Stack ที่เลือกใช้ (SRS หัวข้อ 17.1 เวอร์ชันง่าย)

- **Frontend:** `HTML5` (แบ่งตามบทบาทและหน้าที่รวม 8 หน้าเว็บ)
- **Styling & Responsive:** `CSS3` (ไฟล์ `css/style.css` ใช้ CSS Variables โทนสี `#112A46`, Flexbox, Grid และ Media Queries)
- **Application Logic:** `Vanilla JavaScript (ES6)` (ไฟล์ `app.js` ควบคุม DOM, URLSearchParams, Event Listeners และ CRUD)
- **Data Storage:** `Web Storage API (localStorage)` จัดเก็บข้อมูลฝั่ง Client ถาวรในเบราว์เซอร์โดยไม่ต้องติดตั้ง Backend

📂 3.2 โครงสร้างไฟล์ของโปรเจกต์ (Project File Structure)

```
helpdesk/
├── index.html                    # หน้าแรก เลือกมุมมอง User / Staff / Admin
├── user.html & ticket-user.html # แจ้งปัญหาใหม่, ดู/แก้ไข/ยกเลิก Ticket ของฉัน
├── staff.html & staff-ticket.html # ดูงานที่ได้รับมอบหมาย, อัปเดตสถานะ/บันทึกผล
├── admin.html & ticket.html      # Dashboard, ค้นหา/กรอง, มอบหมายงาน, ลบ Ticket
├── issue-type.html               # จัดการเพิ่ม/ลบหมวดหมู่ประเภทปัญหา
├── app.js                        # โค้ด JavaScript ควบคุม Logic และ CRUD ทั้งหมด
└── css/style.css                 # โค้ดตกแต่ง UI, Status Badges และ Responsive
```

3.3 โครงสร้างข้อมูลที่จัดเก็บใน LocalStorage (Data Model — ตาม SRS หัวข้อที่ 10)

ระบบจัดเก็บข้อมูลลงใน `localStorage` ผ่าน 3 คีย์หลัก ได้แก่ `"tickets"` (Array ของออบเจกต์ Ticket), `"ticketNumber"` (เลขลำดับถัดไปสำหรับสร้างรหัส `JJK-XXXX`) และ `"issueTypes"` (Array ของหมวดหมู่ปัญหา) โดยมีโครงสร้าง Field ดังนี้:

ชื่อ Field (`app.js`)	ชนิดข้อมูล	ตัวอย่างค่าข้อมูลจริง	คำอธิบายและบทบาทในระบบ
`ticketId`	String	`"JJK-0001"`	รหัสอ้างอิง Ticket ที่สร้างอัตโนมัติ (ใช้เป็น Primary Key อ้างอิงผ่าน URL `?id=...`)
`requesterName`	String	`"นายรัตนภัทร นันทะแสน"`	ชื่อ-นามสกุลของผู้แจ้งปัญหา (บังคับกรอก มี Validation ตรวจสอบห้ามว่าง)
`requesterEmail`	String	`"rattanaphat.n@ksu.ac.th"`	อีเมลติดต่อของผู้แจ้งปัญหา (หากไม่ระบุ ระบบจะแสดงค่า `"-"`)
`title`	String	`"เครื่องคอมพิวเตอร์เปิดไม่ติด หน้าจอดำ"`	หัวข้อปัญหาโดยย่อ (บังคับกรอก ใช้แสดงในการ์ดรายการและใช้ค้นหาด้วย Keyword)
`description`	String	`"เครื่องแถวที่ 2 เครื่องที่ 5 กดปุ่มเปิด..."`	รายละเอียดอาการของปัญหาอย่างครบถ้วน (บังคับกรอก และใช้ค้นหาด้วย Keyword)
`issueType`	String	`"Computer Hardware"`	หมวดหมู่ประเภทปัญหา (ดึงค่าแบบไดนามิกจาก `localStorage.getItem("issueTypes")`)
`location`	String	`"ห้องปฏิบัติการคอมพิวเตอร์ Lab 1"`	สถานที่หรือห้องที่เกิดปัญหา (บังคับกรอก มี Validation ตรวจสอบห้ามว่าง)
`priority`	String	`"Low" | "Medium" | "High" | "Urgent"`	ระดับความเร่งด่วนของปัญหา 4 ระดับ ใช้สำหรับกรองข้อมูลและสรุปยอดบน Dashboard
`status`	String	`"Open" | "In Progress" | "Resolved" | "Cancelled"`	สถานะของ Ticket (เริ่มต้นเป็น `"Open"` เสมอ แสดงผลด้วย Badge สีแยกตามสถานะ)
`assignedTo` / `assignedAt`	String	`"IT Staff 1"` / `"2026-10-18T10:10Z"`	ชื่อเจ้าหน้าที่ผู้รับผิดชอบ (`""` = ยังไม่มอบหมาย) และวันเวลาที่มอบหมายงาน
`resolutionNote`	String	`"ดำเนินการลงโปรแกรมใหม่เรียบร้อยแล้ว"`	บันทึกหมายเหตุการดำเนินงานหรือวิธีการแก้ไขปัญหาโดย Staff หรือ Admin
`createdAt` / `updatedAt`	ISO String	`"2026-10-18T09:15:00.000Z"`	วันเวลาที่สร้าง Ticket และวันเวลาที่แก้ไขล่าสุด แปลงเป็น พ.ศ. ด้วย `formatThaiDate()`

3.4 สถานะของ Ticket, ระดับความสำคัญ และ Workflow การทำงาน (ตาม SRS หัวข้อ 11–15)

1. แจ้งปัญหาใหม่ (User)
กรอกฟอร์มใน `user.html`
ตรวจ Validation ครบถ้วน

➔

2. สถานะ Open
สร้างรหัส `JJK-XXXX`
User แก้ไข/ยกเลิกยังได้

➔

3. มอบหมายงาน (Admin)
Admin เลือกผู้รับผิดชอบ (`IT Staff 1` / `2` / `Admin`)

➔

4. In Progress (Staff)
Staff รับงาน เปลี่ยนสถานะ
และบันทึก `resolutionNote`

➔

5. Resolved / Cancelled
ซ่อมเสร็จเปลี่ยนเป็น `Resolved`
อัปเดตสถิติขึ้น Dashboard

🏷️ สถานะทั้ง 4 สถานะ (เวอร์ชันง่ายตาม SRS หัวข้อที่ 12)

- **Open** — เปิด Ticket ใหม่ รอเจ้าหน้าที่รับเรื่อง (User แก้ไข/ยกเลิกได้)
- **In Progress** — เจ้าหน้าที่รับเรื่องและกำลังดำเนินการแก้ไข
- **Resolved** — ดำเนินการแก้ไขปัญหาและบันทึกผลเรียบร้อยแล้ว
- **Cancelled** — ยกเลิกรายการแจ้งปัญหา (โดยผู้แจ้งหรือผู้ดูแลระบบ)

⚡ ระดับความสำคัญ 4 ระดับ (Priority ตาม SRS หัวข้อที่ 13)

- **Low** — ปัญหาเล็กน้อย ไม่เร่งด่วน (เช่น เมาส์ไร้สายคลิกไม่ติด)
- **Medium** — ปัญหาทั่วไป ดำเนินการเมื่อมีเวลา (เช่น โปรแกรมเปิดไม่ได้)
- **High** — ส่งผลกระทบต่อการเรียนหรือการทำงาน (เช่น เครื่องพิมพ์ติด)
- **Urgent** — ปัญหารุนแรง ต้องเร่งแก้ไขทันที (เช่น เน็ตห้อง Lab ล่มทั้งห้อง)

4. คำอธิบายโค้ดและตรรกะหลักของระบบจากไฟล์ `app.js` (Core Code Logic)

📌 **อ้างอิงข้อกำหนด SRS หัวข้อที่ 22 และ 24:** นักศึกษาต้องสามารถอธิบายโครงสร้างระบบและโค้ดหลักของโปรเจกต์ได้ด้วยตนเอง โดยเฉพาะ **(1) Logic การสร้าง Ticket และรหัสอัตโนมัติ, (2) Logic การเปลี่ยนสถานะและจัดการสิทธิ์** และ **(3) Logic การค้นหาและกรองข้อมูล (Search & Filter)**

4.1 Logic การสร้าง Ticket การตรวจสอบข้อมูล (Validation) และการรันเลข Ticket ID อัตโนมัติ (`app.js` บรรทัดที่ 11–92)

```
let tickets = JSON.parse(localStorage.getItem("tickets")) || [];
let ticketNumber = Number(localStorage.getItem("ticketNumber")) || 1;

ticketForm.addEventListener("submit", function(event) {
    event.preventDefault(); // ป้องกันหน้าเว็บรีเฟรชเมื่อกดปุ่ม Submit
    const requesterName = document.getElementById("requesterName").value;
    const requesterEmail = document.getElementById("requesterEmail").value;
    const title = document.getElementById("title").value;
    const description = document.getElementById("description").value;
    const issueType = document.getElementById("issueType").value;
    const location = document.getElementById("location").value;
    const priority = document.getElementById("priority").value;
    const ticketId = `JJK-${String(ticketNumber).padStart(4, "0")}`; // สร้างรหัส เช่น JJK-0001

    if (requesterName.trim() === "" || title.trim() === "" || description.trim() === "" || location.trim() === "") {
        alert("กรุณากรอกข้อมูลให้ครบ"); return;
    }
    const ticket = {
        ticketId, requesterName, requesterEmail, title, description, issueType, location, priority,
        status: "Open", assignedTo: "", resolutionNote: "",
        createdAt: new Date().toISOString(), updatedAt: new Date().toISOString()
    };
    tickets.push(ticket);
    localStorage.setItem("tickets", JSON.stringify(tickets)); // บันทึกรายการ Ticket ลง LocalStorage
    ticketNumber++;
    localStorage.setItem("ticketNumber", ticketNumber);       // อัปเดตเลขลำดับถัดไปลง LocalStorage
});
```

คำอธิบายการทำงาน: เมื่อผู้ใช้กดส่งฟอร์ม ระบบใช้ `event.preventDefault()` เพื่อสกัดการรีโหลดหน้าเว็บ จากนั้นนำตัวแปร `ticketNumber` มาแปลงเป็นข้อความ 4 หลักด้วยเมธอด `String(ticketNumber).padStart(4, "0")` ได้รหัส `JJK-0001` และตรวจสอบความครบถ้วนของข้อมูลด้วย `.trim() === ""` หากครบถ้วนจึงสร้างออบเจกต์ที่กำหนดสถานะเริ่มต้นเป็น `"Open"` เสมอ แล้วแปลงเป็น JSON String ด้วย `JSON.stringify()` บันทึกลง `localStorage` พร้อมบวกค่า `ticketNumber++` เตรียมไว้สำหรับรายการถัดไป

4.2 Logic การเปลี่ยนสถานะ การมอบหมายงาน และการจำกัดสิทธิ์แก้ไข/ยกเลิกของ User (`app.js` บรรทัดที่ 228–293, 763–900)

```
// 1) การเปลี่ยนสถานะและมอบหมายงานโดย Admin / Staff (อัปเดตเวลา updatedAt และ assignedAt อัตโนมัติ)
saveStatus.addEventListener("click", function() {
    ticket.status = document.getElementById("statusSelect").value;
    ticket.updatedAt = new Date().toISOString();
    localStorage.setItem("tickets", JSON.stringify(tickets));
    alert("เปลี่ยนสถานะเรียบร้อย"); location.reload();
});
saveAssigned.addEventListener("click", function() {
    const newAssigned = document.getElementById("assignedSelect").value;
    if (ticket.assignedTo !== newAssigned) {
        ticket.assignedTo = newAssigned;
        ticket.assignedAt = newAssigned ? new Date().toISOString() : "";
        ticket.updatedAt = new Date().toISOString();
    }
    localStorage.setItem("tickets", JSON.stringify(tickets)); location.reload();
});
// 2) กฎการจำกัดสิทธิ์ฝั่ง User: แก้ไขหรือยกเลิกได้เฉพาะเมื่อสถานะยังเป็น "Open" เท่านั้น
if (ticket && ticket.status !== "Open") {
    editUserTicket.style.display = "none";
    cancelUserTicket.style.display = "none";
}
```

คำอธิบายการทำงาน: ระบบดึงรหัส Ticket จาก URL ด้วย `new URLSearchParams(window.location.search).get("id")` แล้วค้นหาออบเจกต์ด้วย `tickets.find(...)` เมื่อ Admin หรือ Staff กดบันทึกสถานะหรือมอบหมายงาน ระบบจะอัปเดตค่าในออบเจกต์พร้อมประทับเวลาแก้ไขล่าสุด (`updatedAt`) และเวลามอบหมายงาน (`assignedAt`) ลงใน `localStorage` ทันที ส่วนในฝั่ง User (`ticket-user.html`) ระบบตรวจสอบเงื่อนไขตามกฎ Business Rule ว่าหาก `ticket.status !== "Open"` (เช่น เจ้าหน้าที่รับเรื่องเป็น `In Progress` แล้ว) ระบบจะซ่อนปุ่มแก้ไขและปุ่มยกเลิกทันทีเพื่อป้องกันการแก้ไขข้อมูลระหว่างที่เจ้าหน้าที่กำลังปฏิบัติงาน

4. คำอธิบายโค้ดและตรรกะหลักของระบบ (ต่อ): Search, Filter & Dashboard

4.3 Logic การค้นหาและกรองข้อมูลหลายเงื่อนไขพร้อมกัน (`filterTickets()` ใน `app.js` บรรทัดที่ 342–444)

```
function filterTickets() {
    const keyword = searchInput.value.toLowerCase();
    const selectedType = issueTypeFilter.value;
    const selectedStatus = statusFilter.value;
    const selectedPriority = priorityFilter.value;
    const selectedAssigned = assignedFilter.value;
    const latestTickets = JSON.parse(localStorage.getItem("tickets")) || [];

    const filteredTickets = latestTickets.filter(function(ticket) {
        // 1. ค้นหาจากรหัส Ticket ID, หัวข้อปัญหา หรือรายละเอียดปัญหา (Case-insensitive)
        const matchSearch = ticket.ticketId.toLowerCase().includes(keyword) ||
                            ticket.title.toLowerCase().includes(keyword) ||
                            ticket.description.toLowerCase().includes(keyword);
        // 2. กรองตามประเภทปัญหา สถานะ และความเร่งด่วน (ถ้าเลือก "" คือแสดงทั้งหมด)
        const matchType = selectedType === "" || ticket.issueType === selectedType;
        const matchStatus = selectedStatus === "" || ticket.status === selectedStatus;
        const matchPriority = selectedPriority === "" || ticket.priority === selectedPriority;
        // 3. กรองตามผู้รับผิดชอบ (รองรับการกรองรายการที่ "ยังไม่ได้มอบหมาย")
        const matchAssigned = selectedAssigned === "" ||
            (selectedAssigned === "ยังไม่ได้มอบหมาย" ? ticket.assignedTo === "" : ticket.assignedTo ===
selectedAssigned);

        return matchSearch && matchType && matchStatus && matchPriority && matchAssigned; // AND Logic
    });
    // นำ filteredTickets มาสร้าง DOM แสดงผลใหม่ทันทีเมื่อพิมพ์หรือเปลี่ยน Dropdown
}
```

คำอธิบายการทำงาน: ฟังก์ชัน `filterTickets()` ผูกกับ Event `input` ของช่องค้นหา และ Event `change` ของ Dropdown ทั้ง 4 ตัว เมื่อผู้ใช้พิมพ์ค้นหาหรือเลือกตัวกรอง ระบบจะดึงข้อมูลล่าสุดจาก `localStorage` แล้วใช้เมธอด `.filter()` ตรวจสอบตัวแปรบูลีนทั้ง 5 ตัว (`matchSearch`, `matchType`, `matchStatus`, `matchPriority`, `matchAssigned`) โดยเชื่อมด้วยตัวดำเนินการ `&&` (AND) ทำให้สามารถค้นหาคำสำคัญควบคู่กับการกรองหลายเงื่อนไขพร้อมกันได้แบบ Real-time

4.4 Logic การคำนวณสถิติ Dashboard และการจัดการประเภทปัญหา (`app.js` บรรทัดที่ 461–592, 1209–1298)

```
// 1) ฟังก์ชันอัปเดตตัวเลขการ์ดสรุปสถานะบน Dashboard
function updateDashboard() {
    const latestTickets = JSON.parse(localStorage.getItem("tickets")) || [];
    totalCount.textContent = latestTickets.length;
    openCount.textContent = latestTickets.filter(t => t.status === "Open").length;
    progressCount.textContent = latestTickets.filter(t => t.status === "In Progress").length;
    resolvedCount.textContent = latestTickets.filter(t => t.status === "Resolved").length;
    cancelledCount.textContent = latestTickets.filter(t => t.status === "Cancelled").length;
}
// 2) ฟังก์ชันสรุปจำนวนตามประเภทปัญหา (รวมหมวดหมู่ที่ถูกลบเข้ากับ Other อัตโนมัติ)
issueTypes.forEach(function(type) {
    if (type === "Other") return;
    const count = latestTickets.filter(t => t.issueType === type).length;
    issueTypeSummary.innerHTML += `<p>${type}: ${count}</p>`;
});
latestTickets.forEach(function(ticket) {
    if (!issueTypes.includes(ticket.issueType)) otherCount++;
});
```

คำอธิบายการทำงาน: ฟังก์ชัน `updateDashboard()` และ `updateSummary()` คำนวณจำนวน Ticket จากข้อมูลจริงใน `localStorage` โดยจุดเด่นของโค้ดส่วนสรุปประเภทปัญหาคือ หากผู้ดูแลระบบลบประเภทปัญหาบางอันออกจากหน้า `issue-type.html` ไปแล้ว แต่ยังมี Ticket เก่าที่ใช้ประเภทนั้นค้างอยู่ โค้ดบรรทัด `if (!issueTypes.includes(ticket.issueType)) otherCount++;` จะนำ Ticket เหล่านั้นไปนับรวมในหมวด `Other` ให้อัตโนมัติ ทำให้ผลรวมของจำนวน Ticket ตรงกับยอดรวมทั้งหมดเสมอ

4.5 ฟังก์ชันเสริม: การแปลงวันที่เป็นรูปแบบพุทธศักราช (พ.ศ.) และการเรียงลำดับงานของ Staff

- **ฟังก์ชัน `formatThaiDate(date)` (บรรทัดที่ 1–9):** แปลงค่า ISO Timestamp เป็นรูปแบบ `วัน/เดือน/ปี(พ.ศ.) เวลา` โดยใช้ `d.getFullYear() + 543` และใช้ `padStart(2, "0")` จัดรูปแบบชั่วโมงและนาทีให้อ่านง่าย
- **การเรียงลำดับรายการ Ticket:** ในหน้า User และ Admin ใช้ `.slice().reverse()` เพื่อแสดง Ticket ที่แจ้งเข้ามาใหม่ล่าสุดไว้บนสุด ส่วนในหน้า Staff (`showStaffTickets()` บรรทัดที่ 934–936) ใช้ `.sort((a, b) => new Date(b.assignedAt) - new Date(a.assignedAt))` เพื่อเรียงงานที่เพิ่งได้รับมอบหมายล่าสุดขึ้นมาก่อน

5. ตัวอย่างข้อมูล Ticket ในระบบอย่างน้อย 10 รายการ (Sample Data 10 Tickets)

ตามข้อกำหนดการส่งมอบงาน (SRS หัวข้อที่ 23 ข้อ 6) ได้จัดเตรียมข้อมูลตัวอย่าง Ticket ในระบบจำนวน **10 รายการ** (`JJK-0001` ถึง `JJK-0010`) ที่ครอบคลุมประเภทปัญหาทั้ง 7 หมวดหมู่, ทุกระดับความสำคัญ (`Low`, `Medium`, `High`, `Urgent`), ทุกสถานะ (`Open`, `In Progress`, `Resolved`, `Cancelled`) และกระจายผู้รับผิดชอบครบทุกบทบาท ดังตารางต่อไปนี้:

Ticket ID	หัวข้อและรายละเอียดปัญหา	ประเภทปัญหา	สถานที่	Priority	สถานะ	ผู้แจ้ง / ผู้รับผิดชอบ	หมายเหตุการแก้ไข (Resolution Note)
JJK-0001	**เครื่องคอมพิวเตอร์เปิดไม่ติด หน้าจอดำ** เครื่องแถวที่ 2 เครื่องที่ 5 กดปุ่มเปิดแล้วไฟไม่ติด	Computer Hardware	ห้อง Lab 1 (ตึก IT ชั้น 3)	High	Open	นายรัตนภัทร *(ยังไม่มอบหมาย)*	-
JJK-0002	**อินเทอร์เน็ตและ Wi-Fi ห้อง Lab 2 ใช้งานไม่ได้ทั้งห้อง** เชื่อมต่อ Wi-Fi และสาย LAN ในห้องปฏิบัติการไม่ได้	Network/Internet	ห้อง Lab 2 (ตึก IT ชั้น 3)	Urgent	In Progress	คุณครูวรพจน์ ➔ **IT Staff 1**	ตรวจ Switch ชั้น 3 พบพอร์ต Uplink ค้าง กำลังรีเซ็ตอุปกรณ์
JJK-0003	**โปรแกรม VS Code เปิดไม่ขึ้น ขึ้นแจ้งเตือน Error** ค้างและเด้งออกพร้อมข้อความ Extension Host Terminated	Software/Application	ห้องเรียน 4302	Medium	Resolved	น.ส.กมลชนก ➔ **IT Staff 2**	ถอนการติดตั้งและลง VS Code เวอร์ชันล่าสุดใหม่ พร้อมล้าง Cache
JJK-0004	**เครื่องพิมพ์สำนักงานกระดาษติดและไฟกะพริบสีส้ม** สั่งพิมพ์เอกสารราชการแล้วกระดาษติดค้างภายในเครื่อง	Printer/Scanner	สำนักงานสาขาวิชา ชั้น 2	High	Resolved	น.ส.สุดารัตน์ ➔ **IT Staff 1**	นำกระดาษที่ติดในชุดลูกกลิ้งออก ทำความสะอาด และทดสอบพิมพ์ปกติ
JJK-0005	**โปรเจคเตอร์ห้องเรียน 4201 ภาพเป็นสีเหลืองและกะพริบ** ต่อสาย HDMI แล้วภาพสีเพี้ยนและติดๆ ดับๆ	Classroom/Lab Room	ห้องเรียน 4201	High	In Progress	นายธนกร ➔ **IT Staff 2**	ตรวจพบหัวสาย HDMI หลวม กำลังเบิกสาย HDMI เส้นใหม่มาเปลี่ยน
JJK-0006	**ลืมรหัสผ่านเข้าใช้งานระบบเครือข่ายมหาวิทยาลัย** ไม่สามารถล็อกอิน Wi-Fi และอีเมลได้ แจ้งรหัสหมดอายุ	Account/Login	ห้องสมุดกลาง ชั้น 1	Low	Resolved	นายภูวดล ➔ **Admin**	รีเซ็ตรหัสผ่านเริ่มต้นให้ใหม่และส่งรายละเอียดทางอีเมลสำรองแล้ว
JJK-0007	**เมาส์ไร้สายประจำโพเดียมอาจารย์คลิกไม่ติด** เมาส์ที่โต๊ะบรรยายหน้าห้องขยับเคอร์เซอร์ไม่ไป	Computer Hardware	ห้องบรรยายรวม 4101	Low	Cancelled	นายรัตนภัทร *(ยกเลิกโดยผู้ใช้)*	ผู้แจ้งยกเลิก เนื่องจากเปิดสวิตช์ใต้เมาส์แล้วกลับมาใช้งานได้ปกติ
JJK-0008	**เครื่องปรับอากาศตัวที่ 2 ในห้อง Lab 3 น้ำหยดและไม่เย็น** แอร์มีน้ำหยดลงมาบริเวณโต๊ะวางเครื่องคอมพิวเตอร์	Classroom/Lab Room	ห้องปฏิบัติการ Lab 3	Urgent	In Progress	น.ส.ปิยะดา ➔ **IT Staff 1**	ปิดเบรกเกอร์แอร์ชั่วคราว ย้ายคอมออกจากจุดน้ำหยด และตามช่างแอร์
JJK-0009	**ขอติดตั้งโปรแกรม Node.js และ Git สำหรับวิชาพัฒนาเว็บ** ขอให้ติดตั้งลงเครื่องคอมพิวเตอร์ทุกเครื่องในห้อง Lab 1	Software/Application	ห้อง Lab 1 (ตึก IT ชั้น 3)	Medium	Open	คุณครูวรพจน์ *(ยังไม่มอบหมาย)*	-
JJK-0010	**ปลั๊กพ่วงประจำโต๊ะกลุ่มที่ 4 ชำรุด ไฟไม่เข้า** เสียบสายชาร์จโน้ตบุ๊กแล้วไฟไม่เข้า สวิตช์เปิด-ปิดหลวม	Other	ห้อง Co-Working ชั้น 2	Medium	Open	นายณัฐพงศ์ *(ยังไม่มอบหมาย)*	-

📊 สรุปตามสถานะ (รวม 10 รายการ)

- **Open:** 3 รายการ (`JJK-0001, 0009, 0010`)
- **In Progress:** 3 รายการ (`JJK-0002, 0005, 0008`)
- **Resolved:** 3 รายการ (`JJK-0003, 0004, 0006`)
- **Cancelled:** 1 รายการ (`JJK-0007`)

🗂️ สรุปตามประเภทปัญหา (7 หมวด)

- **Computer Hardware:** 2 รายการ
- **Software/Application:** 2 รายการ
- **Classroom/Lab Room:** 2 รายการ
- **Network / Printer / Account / Other:** หมวดละ 1 รายการ

🔥 สรุปตามระดับความสำคัญ

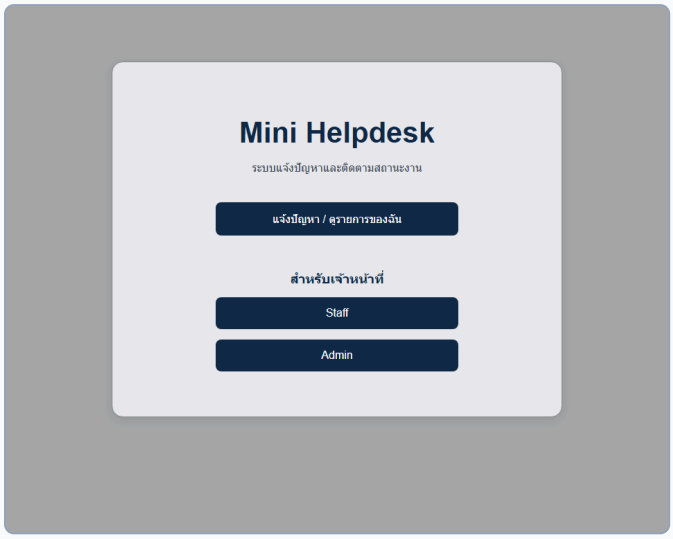
- **Low:** 2 รายการ (`JJK-0006, 0007`)
- **Medium:** 3 รายการ (`JJK-0003, 0009, 0010`)
- **High:** 3 รายการ (`JJK-0001, 0004, 0005`)
- **Urgent:** 2 รายการ (`JJK-0002, 0008`)

6. ภาพหน้าจอหลักของระบบจริง (System Screenshots — ส่วนที่ 1: หน้าหลัก, User และ Admin)

ภาพหน้าจอจากการทำงานจริงของระบบ **SRS Mini Helpdesk** ทั้ง 8 หน้าจอหลักเมื่อบรรจุข้อมูลตัวอย่างทั้ง 10 รายการลงในระบบ:

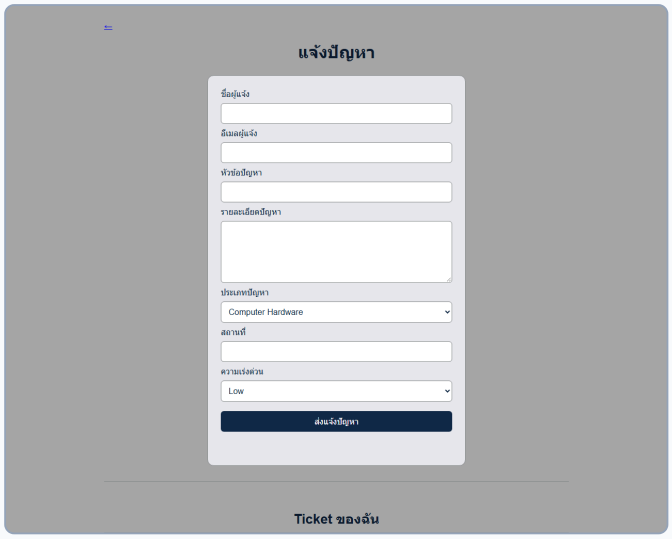
6.1 หน้าหลักของระบบ (`index.html`)

หน้า Home สำหรับเลือกเข้าใช้งานตามบทบาท: ผู้ใช้งานทั่วไป (แจ้งปัญหา/ดูรายการของฉัน), เจ้าหน้าที่ (Staff) และผู้ดูแลระบบ (Admin)



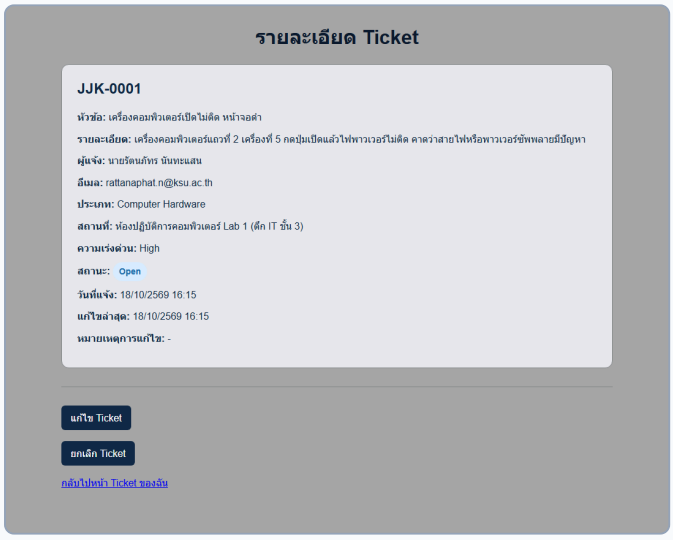
6.2 หน้าแจ้งปัญหาและรายการของฉัน (`user.html`)

แสดงแบบฟอร์มกรอกข้อมูลแจ้งปัญหาใหม่พร้อม Validation และส่วน "Ticket ของฉัน" ด้านล่างที่แสดงสถานะด้วยป้ายสี (Status Badge)



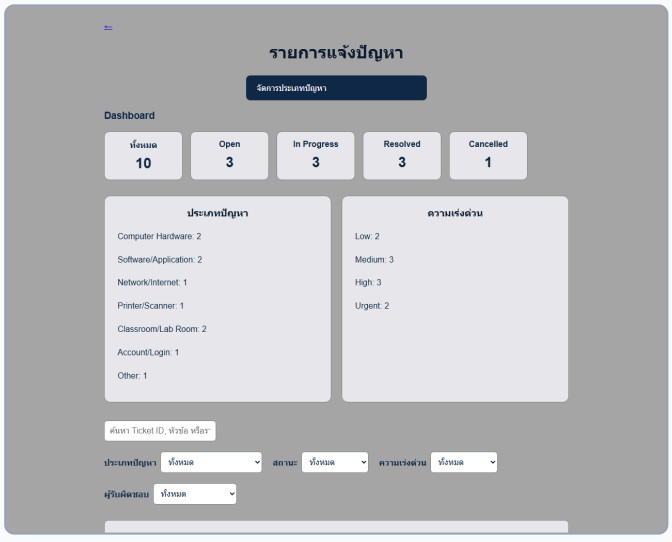
6.3 หน้ารายละเอียด Ticket ฝั่งผู้ใช้ (`ticket-user.html`)

แสดงข้อมูลครบทุก Field ของ `JJK-0001` (สถานะ `Open`) โดยมีปุ่ม "แก้ไข Ticket" และ "ยกเลิก Ticket" ซึ่งจะซ่อนอัตโนมัติเมื่อเจ้าหน้าที่รับงานแล้ว



6.4 หน้า Admin Dashboard และค้นหา/กรอง (`admin.html`)

แสดงการ์ดสรุปยอดรวม (ทั้งหมด 10, Open 3, In Progress 3, Resolved 3, Cancelled 1), สรุปตามประเภทและความเร่งด่วน, ช่องค้นหาและตัวกรอง 4 ด้าน



6. ภาพหน้าจอหลักของระบบจริง (ต่อ): การจัดการของ Admin, Staff และ Responsive UI

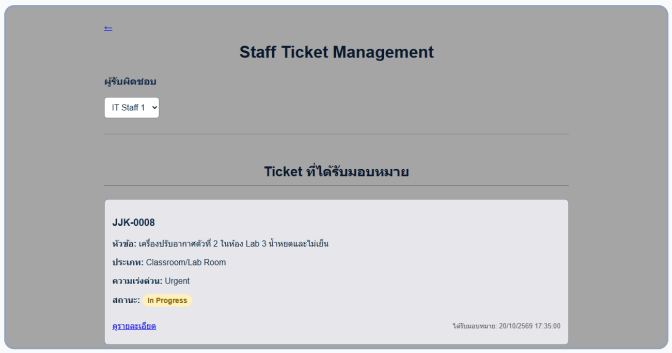
6.5 หน้าจัดการ Ticket ของ Admin (`ticket.html?id=JJK-0002`)

Admin ตรวจสอบรายละเอียด เปลี่ยนสถานะ มอบหมายงาน (`IT Staff 1/2/Admin`) บันทึกหมายเหตุ และลบ Ticket ได้



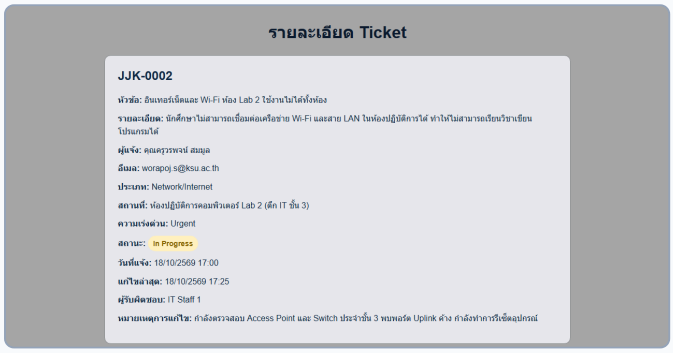
6.6 หน้ารายการงานของเจ้าหน้าที่ (`staff.html`)

Staff เลือกชื่อตนเองจาก Dropdown (เช่น `IT Staff 1`) ระบบจะกรองเฉพาะ Ticket ที่ได้รับมอบหมายและเรียงตามเวลามอบหมายล่าสุด



6.7 หน้าอัปเดตงานของเจ้าหน้าที่ (`staff-ticket.html`)

แสดงรายละเอียดของงานที่ได้รับมอบหมาย พร้อมส่วนควบคุมสำหรับอัปเดตสถานะการซ่อมและบันทึกวิธีการแก้ไข (Resolution Note)



6.8 หน้าจัดการประเภทปัญหา (`issue-type.html`)

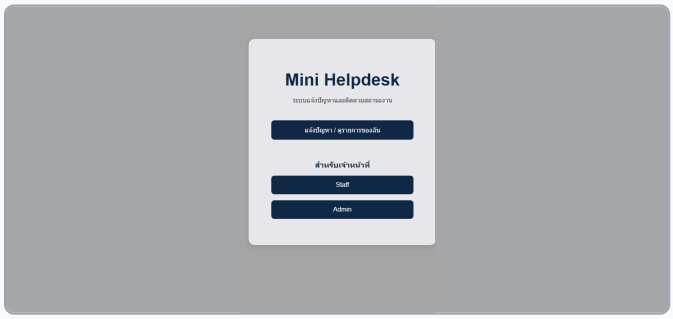
Admin สามารถเพิ่มหมวดหมู่ประเภทปัญหาใหม่หรือลบหมวดหมู่เดิมได้ โดยข้อมูลจะเชื่อมโยงกับตัวเลือกในหน้าแจ้งปัญหาและ Dashboard ทันที



6.9 การรองรับการแสดงผลบนหน้าจอมือถือ (Responsive Design: `@media (max-width: 600px)`)

มุมมอง Mobile หน้าแรก (`index.html`)

การ์ดเมนูหลักและปุ่มกดปรับขนาดเต็มความกว้างหน้าจอ (Width 100%) กดใช้งานบนหน้าจอสัมผัสได้สะดวก



มุมมอง Mobile หน้า Admin Dashboard (`admin.html`)

เมื่อหน้าจอแคบกว่า 600px การ์ด Dashboard จะเรียงซ้อนกันในแนวตั้ง (`flex-direction: column`) และช่อง Input/Select ขยายเต็มความกว้าง



7. สรุปการใช้ AI Tools ในการพัฒนา (ตามข้อกำหนด SRS หัวข้อที่ 22 และ 23 ข้อ 7)

ในการพัฒนาโครงการ **SRS Mini Helpdesk** ผู้จัดทำได้ประยุกต์ใช้เครื่องมือ AI (เช่น ChatGPT / Claude / Antigravity AI Assistant) เป็นผู้ช่วยในกระบวนการพัฒนาควบคู่กับการเขียน ปรับแก้ และทดสอบโค้ดด้วยตนเอง โดยมีรายละเอียดการใช้ AI ในแต่ละขั้นตอนดังนี้:

ขั้นตอนการพัฒนา	บทบาทของ AI Tools ที่นำมาช่วย	การตรวจสอบและปรับแก้โดยผู้พัฒนา
1. วิเคราะห์ Requirement & ออกแบบโครงสร้าง	ช่วยสรุปความต้องการจากเอกสาร SRS แบ่งแยกไฟล์ HTML ทั้ง 8 หน้าตามบทบาท (User, Staff, Admin) และวางโครงสร้าง JSON ใน `LocalStorage`	ผู้พัฒนาปรับโครงสร้างหน้าเว็บให้เชื่อมโยงกันด้วย Query Parameter (`?id=JJK-XXXX`) และกำหนดรหัส Ticket เป็น `JJK-0001`
2. พัฒนา Logic หลัก (JavaScript)	ช่วยแนะนำแนวทางการเขียนฟังก์ชันจัดการ Array ใน `localStorage` เช่น `.find()` , `.filter()` , `.map()` , `padStart()` และการดึงค่าจาก `URLSearchParams`	เขียนและทดสอบฟังก์ชันสร้าง Ticket, การเปลี่ยนสถานะ, การมอบหมายงาน (พร้อมเวลา `assignedAt`) และเงื่อนไขการกรองข้อมูล 5 เงื่อนไขด้วยตนเอง
3. การออกแบบ UI, Status Badge & Responsive	ช่วยแนะนำคู่สี CSS Variables โทนสีกรมท่า (`#112A46`), การจัดวางด้วย Flexbox/Grid, สีป้ายสถานะ (Badge) และ Media Queries สำหรับมือถือ	ปรับแต่งระยะห่าง (Padding/Margin) ขนาดตัวอักษร และทดสอบการแสดงผลจริงบนหน้าจอ Desktop และ Mobile
4. การแก้บั๊ก (Debugging) & จัดทำเอกสาร	ช่วยตรวจหาจุดผิดพลาด เช่น กรณีลบประเภทปัญหาแล้วกระทบยอดสรุปใน Dashboard และช่วยจัดระเบียบโครงสร้างเอกสาร `README.md` และรายงานสรุป	เพิ่ม Logic รวมประเภทปัญหาที่ถูกลบเข้ากับหมวด `Other` อัตโนมัติ และทดสอบฟังก์ชันทั้งหมดจนผ่านเกณฑ์ครบทุกข้อ

8. การตรวจรับตามเกณฑ์ (Acceptance Criteria), แผนงาน 14 วัน และคู่มือการใช้งาน

✅ 8.1 ผลการตรวจรับตามเกณฑ์ขั้นต่ำ (SRS หัวข้อที่ 20)

เกณฑ์การตรวจรับ (Acceptance Criteria 13 ข้อ)	ผลตรวจ
1–3. แจ้งปัญหาใหม่, สร้าง Ticket ID (`JJK-XXXX`), บันทึกครบถ้วน	ผ่าน
4–5. แสดงรายการ Ticket และเปิดดูรายละเอียดครบทุกมุมมอง	ผ่าน
6–7. เปลี่ยนสถานะ Ticket และมอบหมายผู้รับผิดชอบได้	ผ่าน
8–9. ค้นหาจากคำสำคัญ/Ticket ID และกรองตามสถานะ/ประเภท/Priority	ผ่าน
10–11. มี Dashboard สรุปจำนวน และมี UI ที่รองรับมือถือ	ผ่าน
12–13. พร้อม Deploy ใช้งานจริง และมี `README.md` อธิบายครบถ้วน	ผ่าน

📅 8.2 สรุปแผนการดำเนินงาน 2 สัปดาห์ (SRS หัวข้อที่ 21)

ช่วงเวลา	การดำเนินงานจริง
Day 1 – 2	วิเคราะห์ SRS, ออกแบบ Data Model และวางโครงสร้างไฟล์ทั้ง 10 ไฟล์
Day 3 – 4	พัฒนาหน้า `user.html` , ระบบบันทึก Ticket และสร้างรหัส `JJK-XXXX`
Day 5 – 7	พัฒนาหน้ารายการ, หน้ารายละเอียด และระบบ Edit / Cancel / Delete
Day 8 – 10	พัฒนาระบบเปลี่ยนสถานะ, มอบหมาย Staff, Search/Filter และ Dashboard
Day 11 – 14	ปรับ Responsive UI, ทดสอบด้วยข้อมูลตัวอย่าง 10 รายการ และทำรายงานส่งมอบ

📖 8.3 สรุปวิธีติดตั้งและใช้งานระบบ (จาก `README.md`) รวมถึงข้อจำกัดและแนวทางพัฒนาต่อ

วิธีติดตั้งและเปิดใช้งาน (Installation & Usage):

1. **เปิดใช้งานโดยตรง:** เนื่องจากพัฒนาด้วย Static HTML/CSS/JS และใช้ `LocalStorage` จึงสามารถดับเบิลคลิกเปิดไฟล์ `index.html` บน Web Browser (Chrome, Edge, Safari) เพื่อใช้งานได้ทันที
2. **เปิดผ่าน VS Code Live Server:** เปิดโฟลเดอร์ `helpdesk` ใน Visual Studio Code แล้วคลิก *Go Live* เพื่อจำลอง Web Server
3. **การ Deploy ขึ้นระบบจริง:** สามารถอัปโหลดขึ้น GitHub Pages, Netlify หรือ Vercel ได้ทันทีโดยไม่ต้องตั้งค่าฐานข้อมูลภายนอก

ข้อจำกัดปัจจุบัน และแนวทางการพัฒนาต่อยอดในอนาคต:

- **ข้อจำกัด:** ข้อมูลจัดเก็บใน `LocalStorage` ของแต่ละเบราว์เซอร์ (ยังไม่แชร์ข้ามเครื่อง) และใช้การจำลองบทบาทผ่านหน้าเมนูแทนระบบ Login จริง
- **แนวทางพัฒนาต่อ:** เชื่อมต่อฐานข้อมูลจริง (เช่น Supabase / Firebase / SQLite), เพิ่มระบบ Authentication แยกสิทธิ์ตามบัญชีผู้ใช้, เพิ่มการแนบรูปภาพประกอบปัญหา และการแจ้งเตือนผ่าน LINE / Email